

Dynamic Hash Table Implementation

Software Architecture Document

Architecture · Testing · Configuration Management

Architecture Layered Monolith C++17 · Two-Level Larson 1988	Testing Unit + Benchmark 94.2% Line Coverage gcovr 7.2	CI/CD GitLab CI/CD Docker gcc:latest 3-Stage Pipeline
---	--	---

1. Application Architecture

1.1 Architectural Style

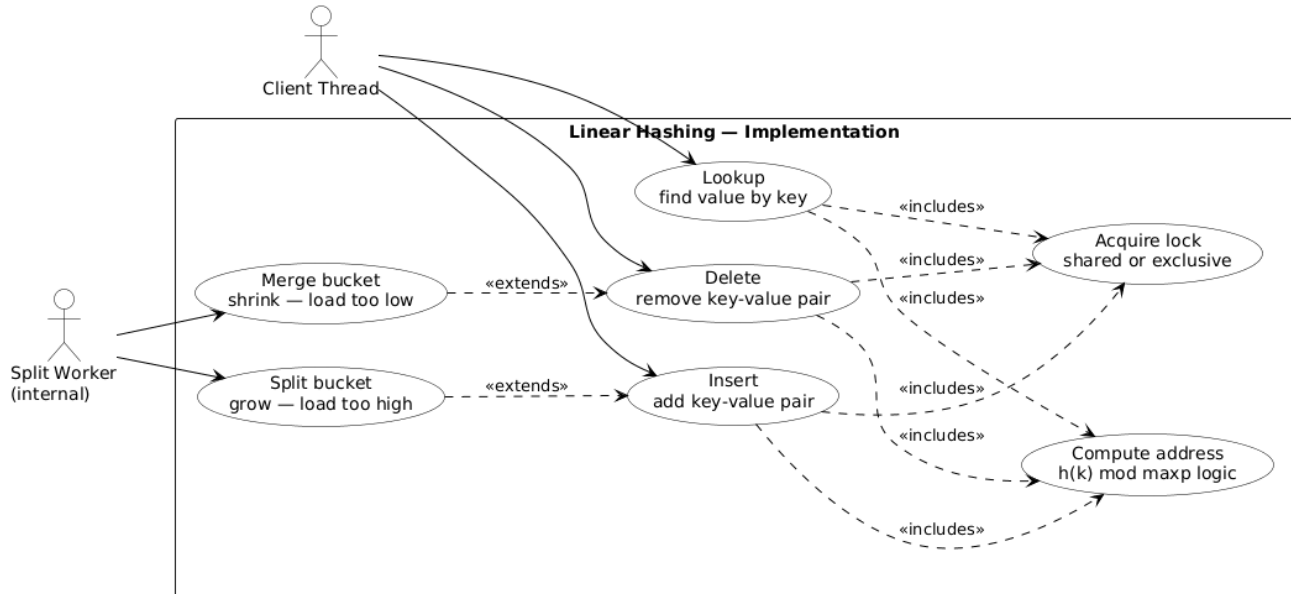
The project follows a **Layered Monolithic Architecture** based on Larson's Two-Level Design (1988). Everything executes inside a single C++ process — no network communication, no separate server, and no microservices. The design achieves concurrency through per-bucket `std::shared_mutex` locking rather than a global lock, enabling fully parallel reads and per-bucket serialised writes.

Architecture type: Single-process, layered monolith with two internal levels — a top-level directory layer for address computation and split/merge logic, and a bottom-level storage layer of independently locked buckets.

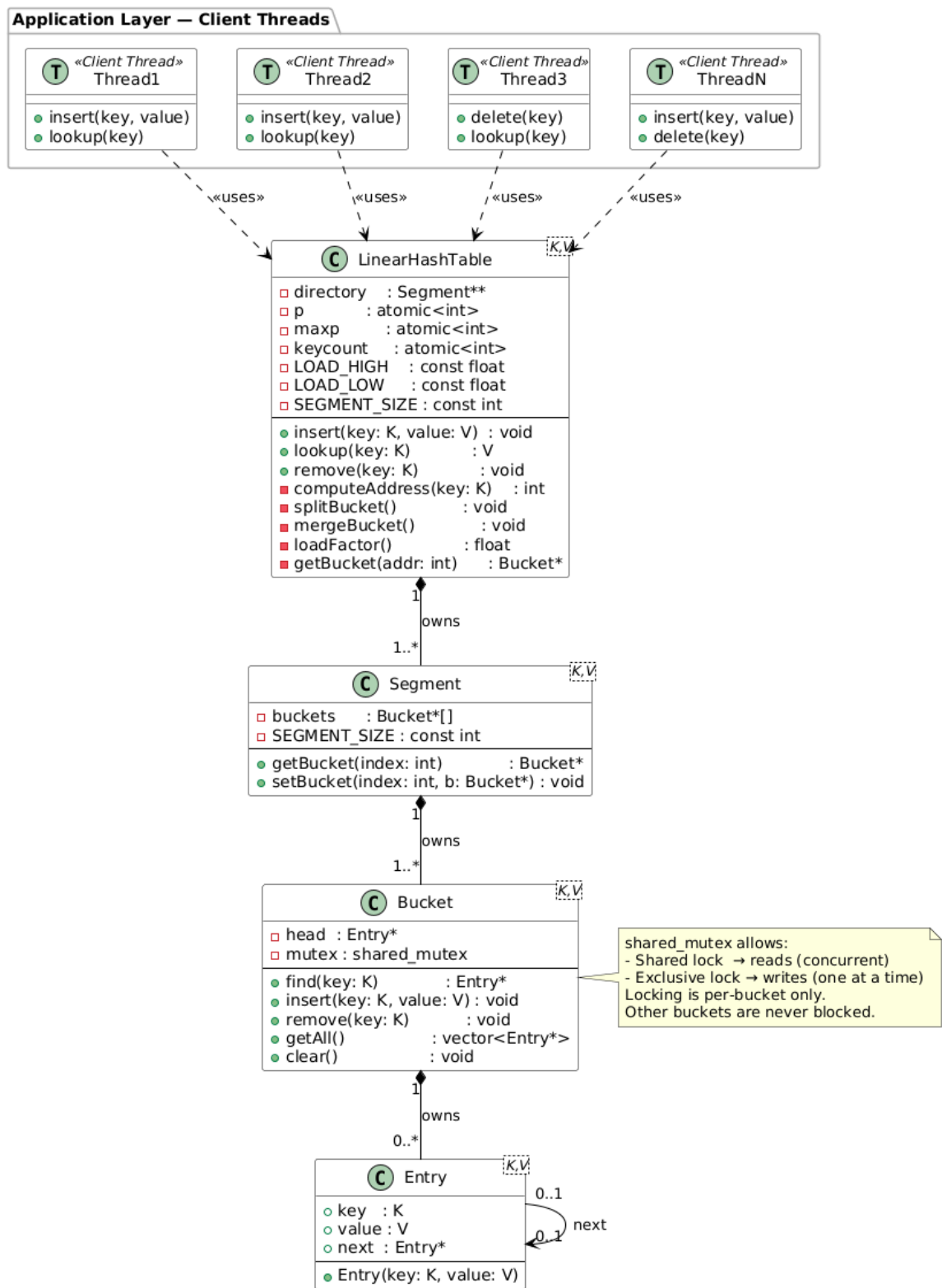
Two Internal Layers

- Application Layer — Multiple client threads call insert, lookup, and delete simultaneously.
- Storage Layer — LinearHashTable owns Segments, which own Buckets, which own Entry linked lists.

Class Hierarchy (Logical View)



Runtime Sequence (Process View)



Three Algorithm Variants Implemented

Algorithm	Based On	Growth Strategy
LinearHashTable	Litwin (1980)	Sequential bucket splits, linear pointer p
ExtendibleHashTable	Fagin et al.	Directory doubling on overflow
LarsonDynamicHashTable	Larson (1988)	Two-level architecture — primary implementation

1.2 Technologies and Tools Used

Language and Standard Library

Technology	Purpose
C++17	Main implementation language
std::shared_mutex	Per-bucket read/write locking — shared for reads, exclusive for writes
std::atomic<int>	Thread-safe counters: p, maxp, keycount
std::chrono	Microsecond-precision benchmarking
std::mt19937	Fixed-seed (42) random key generation for reproducible benchmarks

Build and DevOps Tools

Tool	Role
CMake	Cross-platform build configuration — same CMakeLists.txt on Windows and Linux
GCC (gcc:latest)	C++17 compiler running inside Docker on GitLab CI
Docker	Isolated Linux environment — identical builds regardless of host OS
GitLab CI/CD	Automated pipeline triggered on every git push
gcovr 7.2	Code coverage HTML report generated from GCC gcov data

No external libraries used. The implementation relies entirely on the C++17 standard library. No Boost, no Google Test, no third-party frameworks. The hash table itself is the data structure — no external database is involved.

2. Testing

2.1 Types of Tests

Unit Tests — testBasicOperations()

Basic correctness is verified using C++ built-in `<cassert>` assertions. The test inserts a key, confirms it exists, removes it, and confirms it is gone. If any assertion fails, the program terminates immediately with an error.

```
assert(linear.insert(10) && linear.contains(10)); // insert works
assert(linear.remove(10) && !linear.contains(10)); // delete works

Output:
=== Basic Unit Tests ===
Basic tests passed
```

Functional / Integration Tests — Algorithm Output

All three algorithms are tested together using the same 20 random keys including intentional duplicates and zeros. This verifies cross-algorithm consistency for insert, lookup, and remove operations.

```
Keys: 49 12 7 98 60 65 12 34 49 10 7 0 53 39 64 4 0 88 5 1
      (20 inputs → 16 unique keys — duplicates correctly rejected)

Testing key: 49
Linear Contains:      Yes
Extendible Contains: Yes
Larson Contains:     Yes

After Removal of key 49:
Linear Contains:      No
Extendible Contains: No
Larson Contains:     No
```

Algorithm State Snapshot After 16 Keys Inserted

Algorithm	Buckets	Splits	Load Factor	Level / Depth
Linear Hash Table	11	9	0.727	Level 2, Next Split: 3
Extendible Hash Table	12	10	0.667	Global Depth: 5
Larson Dynamic Hash Table	11	9	0.727	Level 2, Next Split: 3

Performance / Benchmark Tests — 10,000 Keys

10,000 random keys are inserted into each algorithm and then searched 100 times to measure throughput. Results demonstrate significant performance differences between the three approaches.

Algorithm	Unique Keys	Buckets	Splits	Load Factor	Insert Time	Search (100x)
Linear	9,484	3,162	3,158	0.750	5,759 µs	81,968 µs
Extendible	9,484	3,367	3,365	0.704	981,481 µs	1,069,843 µs
Larson	9,484	3,162	3,158	0.750	5,122 µs	77,533 µs

Key Finding: Larson and Linear hashing are approximately 191x faster than Extendible hashing for inserts (5,122 µs vs 981,481 µs) and 13x faster for search. Extendible hashing's directory-doubling strategy incurs significantly higher overhead at scale.

2.2 Testing Tools and Frameworks

Tool	Role	External?
<cassert> (C++17 STL)	Unit test assertions	No — built-in
std::chrono	Microsecond performance measurement	No — built-in
gcovr 7.2	Code coverage HTML report generation	Yes — installed in pipeline
GitLab CI/CD	Automated test execution on every push	Yes — platform

No external testing framework (Google Test, Catch2, Boost.Test) was used. All testing was implemented manually using the C++ standard library only.

2.3 Code Coverage

Coverage was measured using GCC's built-in gcov instrumentation, compiled with --coverage -fprofile-arcs -ftest-coverage flags in Debug mode. Reports were generated by gcovr 7.2.

Overall Coverage Summary

Metric	Executed	Total	Coverage	Status
Line Coverage	404	429	94.2%	High
Function Coverage	40	41	97.6%	High
Branch Coverage	327	554	59.0%	Medium

Per-File Coverage Breakdown

File	Lines	Functions	Branches	Status
include/ExtendibleHashTable.h	100.0%	100.0%	—	Full
main.cpp	100.0%	100.0%	51.6%	Full
src/ExtendibleHashTable.cpp	98.2%	100.0%	63.4%	High
src/LinearHashTable.cpp	97.6%	100.0%	71.7%	High
tests/test_main.cpp	100.0%	100.0%	59.9%	Full
src/LarsonDynamicHashTable.cpp	79.8%	91.7%	53.0%	Medium

Branch coverage (59%) is lower because edge cases such as merge-on-shrink, empty table operations, and overflow error paths are not fully exercised. LarsonDynamicHashTable.cpp at 79.8% line coverage suggests the merge/shrink code path was not triggered during the test run.

2.4 Testing Strategy

When	What
Every git push	GitLab CI automatically triggers the full 3-stage pipeline
Build stage	Code compiled with --coverage debug flags enabled
Test stage	App (algorithm demo) and HashTest (unit + benchmark) both executed
Coverage stage	gcovr reads .gcda runtime files and generates HTML report

Testing is fully automated — no manual steps are required after a push. Every commit triggers build, test, and coverage reporting with results visible in the GitLab pipeline UI.

3. Configuration Management

3.1 Version Control System

Item	Detail
System	Git
Remote Host	GitLab VSB — gitlab.vsb.cz
Repository	jez04-vyuka/swi/swi2026en/pal0343_sad
Branch	master

Repository Structure

```
project-root/
├── .gitlab-ci.yml      ← CI/CD pipeline definition
├── .gitignore          ← excludes build/, *.exe, *.o
├── CMakeLists.txt      ← build configuration
├── main.cpp            ← algorithm demo + benchmarks
├── readme.md
├── include/
│   ├── LinearHashTable.h
│   ├── ExtendibleHashTable.h
│   └── LarsonDynamicHashTable.h
├── src/
│   ├── LinearHashTable.cpp
│   ├── ExtendibleHashTable.cpp
│   └── LarsonDynamicHashTable.cpp
├── tests/
│   └── test_main.cpp
```

.gitignore — Excluded Files

```
build/          ← CMake generated, platform-specific (Windows paths break Linux
build)
*.exe           ← Windows compiled binaries
*.o *.obj       ← compiler object files
*.a *.lib       ← static libraries
.vs/ .vscode/ ← IDE directories
```

3.2 Build System

Tool	Role
CMake	Project configuration and Makefile generation
GCC / g++	C++17 compiler
Unix Makefiles	Generated by CMake on Linux (GitLab CI)

Build Commands Used in Pipeline

```
# Step 1 - Configure project
cmake -S . -B build -G 'Unix Makefiles'
      -DCMAKE_BUILD_TYPE=Debug
      -DCMAKE_CXX_FLAGS='--coverage -fprofile-arcs -ftest-coverage'

# Step 2 - Compile using 2 CPU cores
cmake --build build --parallel 2
```

CMake was chosen because the same CMakeLists.txt works on both Windows (Visual Studio) and Linux (GCC on GitLab CI), making the build fully cross-platform without changes to source files.

3.3 Application Configuration

Configuration is handled through CMake variables and CI/CD environment variables defined in .gitlab-ci.yml. No .env files or external configuration files are used.

Configuration	Where Defined	Value
Build type	.gitlab-ci.yml variable	Debug (for coverage instrumentation)
Coverage flags	CMake CXX flags	--coverage -fprofile-arcs -ftest-coverage
Load factor high	Source code constant	0.75
Load factor low	Source code constant	0.25
Bucket size	Constructor parameter	4 (default)
Random seed	Hardcoded in main.cpp	42 (fixed for reproducible benchmarks)

3.4 CI/CD Pipeline

Platform: GitLab CI/CD running inside Docker (gcc:latest). The pipeline is defined entirely in .gitlab-ci.yml and consists of three sequential stages. Every git push triggers the full pipeline automatically.

Pipeline Stages

Stage	Actions	Output
1. build	Install cmake · Configure with coverage flags · Compile with 2 cores	build/ folder saved as artifact
2. test	Run ./build/App · Run ./build/HashTest · Capture output to report.html	report.html — exposed as 'Full Report'
3. coverage	Install gcovr · Generate HTML coverage report · Fail pipeline if coverage < 70%	coverage-report/ — exposed as 'Coverage Report'

Artifacts Produced by Pipeline

Artifact	Available At
report.html	GitLab → Pipeline → 'Full Report' button
coverage-report/index.html	GitLab → Pipeline → 'Coverage Report' button
Coverage % badge	GitLab → Pipelines list (auto-extracted by regex)

Fully automated: Every git push triggers build → test → coverage with zero manual intervention. The pipeline enforces a minimum 70% line coverage threshold — the job fails if coverage drops below this level.